Dual Stage Text Steganography Using Unicode Homoglyphs

Sachin Hosmani, H.G. Rama Bhat(✉), and K. Chandrasekaran

National Institute of Technology, Mangalore, Karnataka, India
{sachinhosmani,ramabhathg1993}@gmail.com, kchnitk@ieee.org

Abstract. Text steganography is hiding text in text. A hidden text gets hidden in a cover text to produce a plain looking stego text. This plain looking stego text is posted as the message which no one suspects to contain anything concealed. Today, text messages are a common mode of communication over the internet and it is associated with a huge amount of traffic. Steganography is an added layer of protection that can be used for security and privacy. In this paper, we describe a text steganography approach that provides a good capacity and maintains a high difficulty of decryption. We make use of approaches of space manipulation, linguistic translation and Unicode homoglyphs in our algorithm. Our implementation is in Python. Also, we explain a parallel approach for hiding large hidden text messages in large cover text messages.

Keywords: Steganography · Information hiding · Text steganography

1 Introduction

Steganography is the art of writing messages that contain hidden messages in such a way that no one suspects the presence of the hidden messages. While cryptography tries to prevent an adversary from decrypting the contents of a message, steganography tries to prevent the adversary from suspecting the message to be of interest in the first place. When one sends a hidden message concealed in another message, no one will suspect that the outer message contains anything of interest outside its plain meaning. Steganography has had a high prevalence in history, well before the era of computers and internet.

Steganography can be well explained with an example. [1] Two accomplices in crime Alice and Bob are imprisoned and are allowed to communicate only via a monitored communication channel. Alice and Bob want to hatch an escape plan. Obviously, they don't want the prison warden to know about this. They cannot speak in a language foreign to the warden because the warden will suspect something fishy and will block their communication. They can, however speak in plain English, which the warden understands well, but also hide pieces of information in their plain English that the warden couldn't possibly suspect. The general model of Steganography is shown Fig. 1 where the sender embeds the hidden text in the cover text and the receiver deciphers it.

© Springer International Publishing Switzerland 2015
J.H. Abawajy et al. (Eds): SSCC 2015, CCIS 536, pp. 265–276, 2015.
DOI: 10.1007/978-3-319-22915-7_26

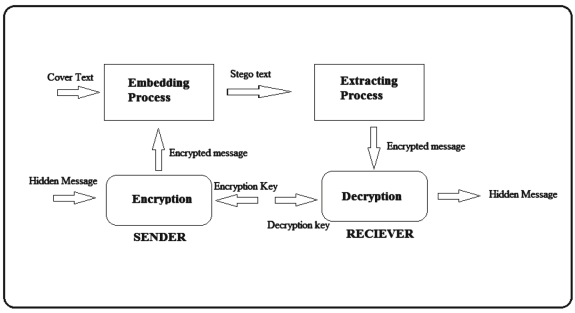


Fig. 1. Steganography model

In this section we introduce the concept of steganography and its applications. In the Related works section, we present different works that have been done in the field of steganography. In the next section we present our approach to steganography with detailed descriptions of our steganography algorithms. Under the Experiments and Results section, we give examples and analyze different steganography algorithms and compare them with our algorithm using different parameters. Finally we conclude and suggest future work that can be done.

2 Literature Survey

People have been hiding information using different methods and variations throughout history. For example ancient Greeks wrote text on wax-covered tablets. To pass a hidden message, a person would scrape wax off a tablet, write a message on the underlying wood and again cover the tablet with wax to make it appear blank and unused [2,3]. During World war 2, invisible inks were used for hiding information. With invisible ink, a seemingly innocent letter could contain a very different message written between the lines [4]. Document layouts are also used for hiding important information. Documents were also marked and identified by modulating the position of lines and words [5].

Text steganography is an important type of steganography that is used. There are many approaches that have been used for text steganography. Some the of popular approaches that have been used for steganography are briefly described here.

Line shift method is a method in which the lines of the cover text are shifted vertically up or down depending on the encountered bits in the hidden text [6]. This method has a lot of limitations. It cannot be used in situations where there is no control over how lines are vertically spaced before rendering. It cannot be used in situations where lines are to be sent as a single string across the medium. This approach also has very low ratio (HCR). Also this approach has a high decoding probability since the number of lines a message might span across is usually less compared to the size of the message.

White steganography is another text steganography method which involves using one or two spaces after every word in the cover text depending on whether the current bit in the hidden text is 0 or 1. This method provides a higher HCR and also provides a lower DP because of the large number of combinations possible for a hidden text [7].

Linguistically-driven generation methods are methods where a context free grammar is written to generate natural like text [10]. Each production in the grammar tells us how to expand a part (or whole) of the message being sent. If a production has multiple possibilities, the choice gives us an opportunity to hide bits. Depending on the choices we make while choosing the productions, the hidden message get covertly written. Since the generated text is completely generated from the grammar, it isn't possible to control it.

Linguistic translation involves replacing certain or all words of the cover text with synonyms so that the meaning of the cover text remains the same but gets modified to contain the secret message [9]. This method offers an HCR that depends on the kind of words in the cover text, but for regular English usage it offers a good HCR. DP is also known to be low.

Unicode is a character set that is internationally recognised as a standard for encoding characters. Since it has to cater to a large number of languages, the set is very large. A character set is a set of code points that unambiguously define characters and their scripts or groups. Code points are rendered as glyphs. A glyph can be thought of as the visual representation of a character. Homoglyphs are glyphs that cannot be quickly identified as different glyphs with visual inspection. Unicode has many homoglyph sets [8]. Unicode homoglyphs can be used in steganography because an adversary will not be able to tell if a character is replaced by its homoglyph or not.

3 Text Steganography

Text steganography is hiding text inside text. Some of the main components that are involved as shown in Fig. 2 are:

- Hidden text: The text to hide.
- Cover text: The outer text to hide the hidden text in.
- Stego text: The text created after hiding the text.
- Encoder: The algorithm that encodes the hidden text inside the cover text.
- Decoder: The algorithm that decodes the hidden text from the stego text.

3.1 Evaluation Criteria

- Hidden text Cover text Ratio (HCR):
  If the number of bits in the hidden text is n1 and the number of bits in the cover text is n2, and the two are chosen such that the cover text is used completely for hiding purposes, HCR = n1 / n2. A greater HCR means greater efficiency.

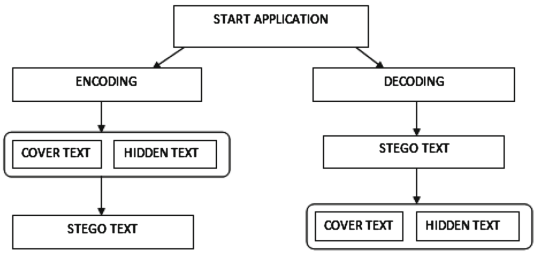


Fig. 2. Flow chart for text steganography

- Difficulty of decoding/Probability of a hidden text guess to be correct (DP: Decoding Probability):
  If the adversary finds out that a message contains a hidden text and knows the approach used for hiding the message, for a message of size constant k, if there are m possibilities of hidden text, then the probability of a correct guess the first time is 1/m. Lower the probability, greater the security of steganography.
- Tamper Resistance:
  An attacker may try to alter the data embedded in a cover, rather than destroy it. A tamper resistant steganographic algorithm makes it extremely hard to alter the hidden data or to erase them and embed a different message.

4 Our Proposed Approach

We decided on an approach that makes use of white steganography (space manipulation), linguistic translation and Unicode homoglyphs. Combining all these approaches into our algorithms has given us a more superior algorithm in terms of HCR and DP.

4.1 Algorithm

There are two algorithms involved. One for producing a steganographed text from a cover text and a hidden text and another for retrieving the hidden text from a steganographed text.

Encoding. (See Algorithm 1) The hidden text is encoded as binary using Huffman coding. This gives us efficiency in number of bits needed to represent the given hidden text. An MD5 checksum of this is found and appended to it with a special delimiter bit combination (specially reserved in the Huffman encoding) in between. It is then encrypted using a key. A simple symmetric stream cipher like XOR can be used for the encryption. This adds an extra layer of security. The cover text is tokenized into spaces and words. Each token in the cover text has to be dealt with depending on whether it is a word or a space. If it is a word, it has to be replaced with a synonym. If it is a space, it has to replaced with two spaces, possibly. Depending on how many words there are in

a synonym list, we can hide some number of bits with this process for words and a constant of 1 bit per space. These many number of bits from the hidden text are extracted from the hidden bit sequence and they are treated collectively as a number and indexed into the synonym list.

Algorithm 1 Encoding

```
Input: Cover text (C), Hidden Text (H), Key (K)
Output: Steganographed text (S)
procedure ENCODE(COVER(C), HIDDEN(H), KEY(K))
    HB = huffman_encode(H)
    HB = HB + delimiter + MD5(HB)
    HB = encrypt(HB, K)
    S = ""
    tokens = tokenize(C)
    while token_count(tokens) != 0 and remaining_bit_count(HB) != 0 do:
        token = get_next_token(tokens)
        if token is a word: then
            synonyms = get_synonyms(token)
            bit_size = largest_bit_size(synonyms.length)
            index = get_next_bits(HB, bit_size)
            synonym = synonyms[index]
            S += synonym
        if token is space: then
            if get_next_bits(HB, 1) == 1: then
                S += "  "
            else
                S += ""
    S += "\a"
    if token_count(tokens) == 0: then
        while token_count(tokens) != 0: do
            token = get_next_token(tokens)
            s += token
    if remaining_bit_count(HB) != 0: then
        for c, i in S: do
            if remaining_count(HB) == 0: then
                break
            homoglyphs = get_homoglyphs(c)
            bit_size = largest_bit_size(homoglyphs.length)
            index = get_next_bits(HB, bit_size)
            homoglyph = homoglyphs[index]
            S[i] = homoglyph
        S[i] = "\a"
    return S
```

For spaces, if the next bit is 1, a pair of spaces are substituted for a space, if the next bit is 0, a single space is kept. At the end, we append a '\a' character. While decoding, this will help us know when to stop. If at the end of this process,

all the bits are hidden, the process is done. Otherwise, for every character in our stego text, we find a list of Unicode homoglyphs for it. Like with synonyms, we choose a homoglyph depending on the bit sequence. We replace the character with the chosen homoglyph. Again, at the end of process, we append a '\a' character to mark that the hiding of bits has ended in the second phase. If we are going through the second phase, it means the first phase completely used the cover text and hence the first '\a' will be only at the end of the stego text. So, the second '\a' will always be before the first one. '\a' is a non-printable character and hence will not affect the visual output of the text.

Decoding. (See Algorithm 2) If the last character of the stego text is "\a", it means that both stages of steganography were involved, the outer stage being Unicode homoglyphs. So, first we have to retrieve bits hidden using Unicode homoglyphs. For every character in the stego text until "\a" is encountered, we find the list of its homoglyphs and find its position in the list. This position tells us the bit sequence hidden in it. This position is converted into binary and appended to H2. At the end of this iteration, we'll have the bit sequence hidden using Unicode homoglyphs. After this stage, we tokenize the stego text as during encoding and iterate through it until "\a" is found. For every token, if it is a space, we check if the next token is also a space. If it is, that means there are two consecutive spaces and this means that the bit hidden is 1. If not, the bit hidden is 0. If we encounter a word, we find the list of its synonyms. We then find the position of this word in the list. This position tells the bit sequence hidden in the word. The position is converted to binary and appended to H1. At the end of this process, we have the bit sequence hidden with the first stage of steganography. Now we have to combine the two pieces of bit sequences to get the complete bit sequence. This is an encrypted bit sequence. We have to decrypt this using the same symmetric cipher and key we used for encryption. Converting this decrypted sequence to characters using Huffman decoding gives us back the original hidden text.

Parallel Processing of Large Texts. (See Algorithm 3) This algorithm tries to allow parallel processing of large strings. The first step is to split the cover text into chunks of MAX_SIZE and processing them individually and parallely. The problem here is that, a chunk may get modified after synonym substitution and space manipulation to contain greater characters than the allowed limit. Hence we need to expand each chunk to the maximum possible size it can take. We are essentially finding an upper bound.

Once we have an upper bound for each chunk's size, we again split the cover text with its expanded state in mind. Each chunk we create will always be containable in a chunk because it is an upper bound. Since we have now partitioned the cover text, we just have to partition the hidden text. It can be simply done by finding the next sequence of hidden text (in bits) that can be hidden in the chunk. We need to consider a lower bound for this, because we can't say beforehand, what synonyms will get used and hence how many bits we can hide using Unicode homoglyphs. Once we have the cover text and hidden text bits split into chunks like this, we can simple parallely process pairs of them for steganography.

Algorithm 2 Decoding

```
Input: Steganographed Text (S), Key (K)
Output: Hidden text (H)
procedure DECODE(STEGANOGRAPHED TEXT(S),KEY(K))
    H1 = H2 = ""
    if S[S.length - 1] == "\a": then
        for each c in S: do
            if c == "\a": then
                break
            homoglyphs = get_homoglyphs(c)
            index = homoglyphs.index_of(c)
            binary_index = number_to_binary(index)
            H2 += binary_index
    tokens = tokenize(S):
    for each token in tokens: do
        if token == "\a": then
            break
        if token == "": then
            next_token = get_next_token(tokens)
            if next_token == "": then
                H1 += "1"
            else
                push_back_token(tokens, next_token)
                H1 += "0"
        if token is a word: then
            synonyms = get_synonyms(token)
            index = synonyms.index_of(token)
            binary_index = number_to_binary(index)
            H1 += binary_index
    HB = H1 + H2
    HB = decrypt(HB, K)
    checksum = HB.split(delimiter)[1]
    if checksum != MD5(HB.split(delimiter)[0]): then
        discard_message()
        return null
    H = huffman_decode(HB)
    return H
```

4.2 Synonym Storage and Retrieval

In real languages, the set of synonyms is not a closed set. Since the semantics of a language are subjective, there is a lot of grayness involved in interpreting the closeness of two words. When we decide to replace a word in the cover text with a synonym, we must ensure that to decode it successfully afterwards, the synonym set we chose the synonym from, is a closed set.

Algorithm 3 Parallel processing for large cover and hidden text

```
1: Input: Cover text (C), Hidden text (H), MAX SIZE: the maximum chunk size
2: Output: Array of cover text index (Ci), hidden text index (Hi) tuples
3: procedure SplitTexts(Cover text (C), Hidden text (H), MAX SIZE)
4:     chunk_count = C.length / MAX_SIZE
5:     chunks = []
6:     for i from 0 to chunk_count - 1 do
7:         chunks.push(C.substr(i*MAX_SIZE, MAX_SIZE))
8:     syn_size_cache = {}
9:     for each chunk in chunks in parallel: do
10:        for each word in chunk: do
11:            synonyms = get_synonyms(chunk)
12:            max_syn_size = 0
13:            for each synonym in synonyms: do
14:                if max_syn_size <synonym.length: then
15:                    max_syn_size = synonym.length
16:            syn_size_cache[word] = max_syn_size
17:    running_size = 0
18:    new_chunks = []
19:    running_chunk = ""
20:    for each word in C: do
21:        running_size += syn_size_cache[word] + 2
22:        if running_size >MAX_SIZE: then
23:            new_chunks.push(running_chunk)
24:            running_size = 0
25:            running_chunk = word
26:        else
27:            running_chunk += word
28:    for each chunk in new_chunks in parallel: do
29:        chunk_capacity[chunk] = get_lower_bound(chunk)
30:    tuples = []
31:    hidden_text_start = 0
32:    chunk_start = 0
33:    for each chunk in chunks: do
34:        tuples.push([chunk_start, hidden_text_start])
35:        hidden_text_start += chunk_capacity[chunk]
36:        chunk_start += chunk.length
37:    return tuples
```

This is our approach when we need to search for a synonym of a word:

- Check if plain text file with that word as title exists
- If it does (each line of it is a synonym belonging to that synonym set):
  - Read the synonym at the required index (line number in the file)
- If it doesn't (this word is a new one encountered):
  - Consult a dictionary for a list of synonyms of the word.
  - For every synonym, create a file with the synonym as the title.

- Populate the file with a synonym in each line.
- Read the synonym at the required index.

This way, we ensure that every synonym set is closed.

5 Results and Analysis

In this section we discuss the results measured in terms of the metrics we explained earlier. We also discuss the specifics of implementation of our algorithms.

5.1 Analysis

Here are the criteria that we considered for the analysis of our steganography algorithm.

1. Hidden text Cover text Ratio (HCR)
   A greater HCR implies more text can be hidden in the given cover text.
   HCR for our steganography algorithm is given by:

$$N = N_1 + \log_2(N_2) + (\log_2(N_3) * N_4) \quad \text{(a)}$$

   where,
   N - Average Number of bits that can be hidden per word
   N_1 - Average number of spaces per word
   N_2 - Average number of synonyms per word
   N_3 - Average number of homoglyphs per character
   N_4 - Average number of characters per word

   This follows from the fact that there can be one bit hidden for per space, $log(N_2)$ bits hidden for a word with N_2 synonyms and $log(N_3) * N_4$ bits hidden for a word with N_4 characters and N_3 homoglyphs per character.
   Assuming, $N_1 = 1$, $N_2 = 4$, $N_3 = 2$, $N_4 = 5$ we get, N = 8.
   For space manipulation alone, $HCR = N_1$ and for linguistic translation, $HCR = log_2(N_2)$.
   HCR for different algorithms are shown in the Fig. 3. for a large cover text.
2. Decoding Probability (DP)
   Decoding Probability is the probability that an adversary guesses the hidden text, if he knows the approach used for hiding the message and also knows that the message contains the hidden message. As DP decreases, the security of the steganography increases.
   If a space was used for hiding text, then there were only 2 possibilities of hidden text, i.e. “0” or “1”. If synonym list was used then there are as many possibilities as there are words in the given synonym set. For e.g. If a set has 4 words, then no. of possibilities are 4.

So in our algorithm which uses the combination of these two along with homoglyphs the possibilities will be the product of possibilities of these two and the homoglyphs.
DP for our steganography algorithm is given by:

$$DP = 1/(X * Y * Z) \tag{b}$$

where,

$$X = 2 \tag{1}$$

$$Y = \prod_{i=1}^{Nw} x_i \tag{2}$$

$$Z = \prod_{i=1}^{Nc} k_i \tag{3}$$

where,
DP - Decoding Probability
X - Number of possibilities for spaces
Y - Number of possibilities for synonyms
Z - Number of possibilities for homoglyphs
Nw - Number of words
Nc - Number of characters
x_i - Number of synonyms for i_th word
k_i - Number of homoglyphs for i_th character

This is simply one divided by the number of possibilities. Each component of our algorithm provides a certain number of possibilities and the total is the product of all of them. For the semantics of space, the adversary has to make just 2 guesses. For synonyms, for each word in the text, the number of synonyms of that word have to be multiplied to get the total number of possibilities. The same logic applies to homoglyphs too.

For space manipulation alone, $DP = 2$
For linguistic translation alone, $DP = Y$
DP for different algorithms are shown in the Fig. 4. for a large cover text.

3. Tamper Resistance
As per our algorithm, in our approach, we find a checksum of our hidden text bits and append it to the hidden text with a delimiter in between. The delimiter is a special bit combination reserved in our Huffman coding. If the message is intercepted in between and is tampered with, it will be impossible for the adversary to tamper the message and yet keep the checksum correct. Note that checksum is done after encryption and so that is an additional level of protection. Also, since the adversary will not have access to the homoglyph

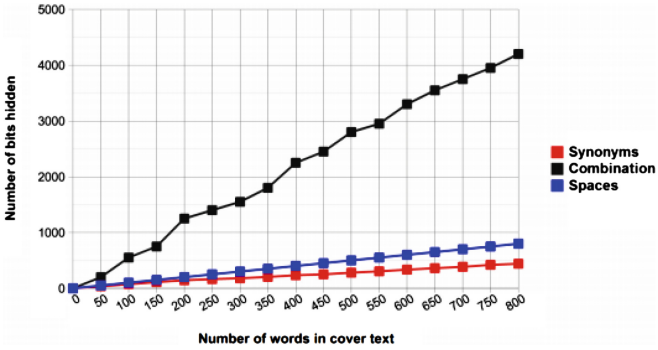


Fig. 3. HCR for different algorithms

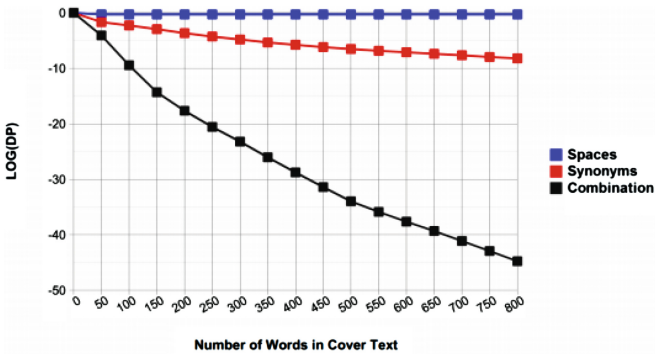


Fig. 4. DP for different algorithms

index table or the synonym table, he will only be able to tamper the message by making random modifications which will necessarily get detected due to the checksum.

Table 1 summarizes comparison of our algorithm with the rest of the algorithms.

Table 1. Comparison of different steganography techniques

Criteria	White	Linguistic	Combination
HCR	1	1	8
DP	Very High	Medium	Very low

6 Conclusion and Future Work

In this paper, we have explained an approach for text steganography. We have discussed the various metrics by which we have evaluated our algorithm.

One of the main components of our algorithm is synonym replacement. The quality of synonyms we get is directly dependent on the dictionary we use. We used Wordnik(Python) for our implementation. A word may have multiple synonyms depending on the context of the sentence. So, we can do natural language processing and try to guess the context in which a word was used. This will reduce the semantic errors we may get due to a wrong choice of synonyms. Also, the frequency of synonym and homoglyph substitutions can be controlled in order to preserve the originality of the cover text to the desired extent.

Our work is limited to an ASCII character set containing alphabets, numbers and special characters. We can extend this to a larger character set, but we need to address the availability of synonyms for these words. Also since our algorithm works for English alone, a suitable internationalization can be carried out.

References

1. Nechta, I., Fionov, A.: Applying statistical methods to text steganography (2011). arXiv preprint arXiv:1110.2654
2. Kahn, D.: The Codebreakers. Macmillan, New York (1967)
3. Norman, B.: Secret Warfare. Acropolis Books, Washington, D.C. (1973)
4. Zim, H.S.: Codes and Secret Writing. William Morrow, New York (1948)
5. J. Brassilet et al., Document Marking and Identification using Both Line and Word Shifting. In: Proceedings of Infocom 1995. IEEE CS Press, Los Alamitos (1995)
6. Shahreza, M.H.S., Shahreza, M.S.: A new synonym text steganography. In: International Conference on Intelligent Information Hiding and Multimedia Signal Processing, pp. 1524–1526 (2006)
7. Bender, W., Gruhl, D., Morimoto, N., Lu, A.: Techniques for data hiding. IBM Syst. J. **35**, 313–336 (1996)
8. Roshanbin, N., Miller, J.: Finding homoglyphs - a step towards detecting unicode-based visual spoofing attacks. In: Bouguettaya, A., Hauswirth, M., Liu, L. (eds.) WISE 2011. LNCS, vol. 6997, pp. 1–14. Springer, Heidelberg (2011)
9. Chapman, M., Davida, G.I., Rennhard, M.: A practical and effective approach to large-scale automated linguistic steganography. In: Davida, G.I., Frankel, Y. (eds.) ISC 2001. LNCS, vol. 2200, pp. 156–165. Springer, Heidelberg (2001)
10. Chapman, M., Davida, G.: Hiding the hidden ciphertext in innocuous text. In: Han, Y., Quing, S. (eds.) ICICS 1997. LNCS, vol. 1334. Springer, Heidelberg (1997)